

**RAJSHAHI UNIVERSITY OF
ENGINEERING & TECHNOLOGY**
Department of Computer Science and Engineering

**Retro Pong Challenge: A Python-Based
Arcade Game with Unit Testing**

A Software Engineering Sessional Project Report

Submitted by:

Md Naim Parvez

Roll No: 2003015

Section: A

Third Year (Even Semester)

Submitted to:

Emrana Kabir Hashi

Assistant Professor

Department of Computer Science and Engineering

Rajshahi University of Engineering & Technology (RUET)

Course Code: CSE 3206

Course Title: Software Engineering Sessional

April 2025

Contents

Abstract	1
1 Introduction	1
1.1 About the Project	1
1.2 Objectives	1
1.3 System Requirement Analysis	2
2 System Design and Development	2
2.1 Architecture Overview	2
2.2 Complete Source Code Implementation (Conceptual)	3
2.2.1 Main Game Implementation (<code>pong_game.py</code>)	3
2.3 Comprehensive Unit Testing Implementation (Conceptual)	7
2.3.1 Complete Test Suite (<code>test_pong_game.py</code>)	7
2.4 Testing Methodology and Results	11
2.4.1 Test Case Analysis	11
2.4.2 Test Execution and Coverage	12
2.5 Test Results	12
3 Project Screenshots (Conceptual)	13
4 Conclusion and Future Scope	14
4.1 Discussion and Conclusion	14
4.2 Testing Achievements	15
4.3 Scope for Further Developments	15
4.3.1 Technical Enhancements	15
4.3.2 User Experience Improvements	16
4.3.3 Advanced Features	16

Abstract

Retro Pong Challenge is a Python-based arcade game developed as a university project at Rajshahi University of Engineering & Technology (RUET). Built using Tkinter for the GUI, the game recreates the classic Pong experience, offering a simple yet engaging interface with distinct paddles and a dynamic ball. Key features include two-player control, real-time score tracking, and a game reset mechanism. The project incorporates comprehensive unit testing using Python's `unittest` framework with 9 distinct test cases to verify the core game logic, including ball movement, collision detection, and scoring, ensuring robust functionality. This report details the development process, including system design, a conceptual overview of the source code implementation, the comprehensive testing methodology, and anticipated results. The conceptual source code is available on GitHub at <https://github.com/NaimParvez/Ruet-CSE/tree/3rd-Year-Even-Sem/Software%20Engineering>. The report also discusses the project's outcomes and potential for future enhancements, such as adding an AI opponent and advanced gameplay modes.

1 Introduction

1.1 About the Project

Retro Pong Challenge is a desktop arcade game developed to provide an entertaining and nostalgic gaming experience. Initiated in 2025, the project utilizes Python's Tkinter library to create a straightforward and responsive interface. It features two-player gameplay where each player controls a paddle, score tracking, and the ability to restart the game. The game involves a ball moving across the screen, bouncing off walls and paddles, with players attempting to hit the ball past their opponent's paddle to score points.

1.2 Objectives

The primary objectives of Retro Pong Challenge are:

- To develop an interactive Pong game with a clean, retro-inspired UI.
- To implement keyboard-based controls for two players to maneuver their respective paddles.
- To accurately track scores and declare a winner when a predefined score is reached.
- To provide options to reset the game for a new match.
- To apply comprehensive unit testing to verify the core game logic, including ball physics, collision detection, and scoring mechanisms.
- To ensure the code is well-structured, maintainable, and allows for future enhancements.
- To demonstrate software testing principles and best practices in a practical application.

1.3 System Requirement Analysis

The system requirements were formulated based on the classic Pong game mechanics and discussions regarding project scope. The requirements include:

- **Functional Requirements:**

- Independent vertical movement of two paddles via keyboard input.
- Continuous ball movement with realistic bounces off top/bottom walls and paddles.
- Accurate scoring when the ball passes an opponent's paddle.
- Declaration of a winner upon reaching 7 points.
- Game reset functionality to start a new match with scores at zero.
- Comprehensive unit tests covering all core game logic methods.

- **Non-Functional Requirements:**

- Responsive and clear UI with a retro arcade aesthetic (e.g., black background, white/colored elements).
- Maintainable and modular code structure.
- 100% test coverage for game logic methods.

- **Hardware Requirements:**

- A computer with at least 2GB RAM and a 1GHz processor.

- **Software Requirements:**

- Python 3.10+ with Tkinter support.
- `unittest` framework for testing.
- A development environment such as Visual Studio Code.

2 System Design and Development

2.1 Architecture Overview

The system is designed with a modular architecture, separating game logic from the user interface:

- **PongGameLogic Class:** Encapsulates all game logic, including managing ball position and velocity, paddle positions, collision detection with walls and paddles, score updates, and game state (e.g., determining a winner).
- **PongGameGUI Class:** Manages the GUI components using Tkinter, handles user input (keyboard events for paddle movement), and orchestrates the visual representation of the game, including drawing paddles, the ball, and the scoreboard.
- **Test Suite:** A comprehensive set of unit tests to validate the functionality of the `PongGameLogic` class.

2.2 Complete Source Code Implementation (Conceptual)

This section provides a conceptual outline of the main game implementation.

2.2.1 Main Game Implementation (pong_game.py)

```
1 # Conceptual Outline for pong_game.py
2 import tkinter as tk
3 import time # Not strictly used in this conceptual GUI logic but common
4
5 # --- Constants ---
6 SCREEN_WIDTH = 800
7 SCREEN_HEIGHT = 600
8 PADDLE_WIDTH = 20
9 PADDLE_HEIGHT = 100
10 BALL_RADIUS = 10
11 PADDLE_SPEED = 30 # Speed of paddle movement
12 BALL_SPEED_X_INITIAL = 5 # Initial X-speed of ball
13 BALL_SPEED_Y_INITIAL = 5 # Initial Y-speed of ball
14 WINNING_SCORE = 7
15
16 # Colors
17 BG_COLOR = '#000000' # Black
18 PADDLE_COLOR = '#FFFFFF' # White
19 BALL_COLOR = '#FFFF00' # Yellow
20 TEXT_COLOR = '#FFFFFF' # White
21
22 # --- Game Logic Class ---
23 class PongGameLogic:
24     def __init__(self):
25         self.ball_pos = [SCREEN_WIDTH / 2, SCREEN_HEIGHT / 2]
26         self.ball_vel = [BALL_SPEED_X_INITIAL, BALL_SPEED_Y_INITIAL]
27         self.paddle1_pos_y = SCREEN_HEIGHT / 2 - PADDLE_HEIGHT / 2
28         self.paddle2_pos_y = SCREEN_HEIGHT / 2 - PADDLE_HEIGHT / 2
29         self.score1 = 0
30         self.score2 = 0
31         self.game_over_flag = False
32
33     def update_ball(self):
34         if self.game_over_flag:
35             return
36
37         self.ball_pos[0] += self.ball_vel[0]
38         self.ball_pos[1] += self.ball_vel[1]
39
40         # Wall collision (top/bottom)
41         if self.ball_pos[1] - BALL_RADIUS <= 0 or \
42             self.ball_pos[1] + BALL_RADIUS >= SCREEN_HEIGHT:
43             self.ball_vel[1] *= -1
44
45         # Paddle collision
46         # Paddle 1 (left)
47         if self.ball_pos[0] - BALL_RADIUS <= PADDLE_WIDTH and \
48             self.paddle1_pos_y < self.ball_pos[1] < self.paddle1_pos_y +
49             PADDLE_HEIGHT:
50             if self.ball_vel[0] < 0: # Check if ball is moving towards
51                 paddle
```

```

50         self.ball_vel[0] *= -1
51
52         # Paddle 2 (right)
53         if self.ball_pos[0] + BALL_RADIUS >= SCREEN_WIDTH - PADDLE_WIDTH
and \
54             self.paddle2_pos_y < self.ball_pos[1] < self.paddle2_pos_y +
PADDLE_HEIGHT:
55             if self.ball_vel[0] > 0: # Check if ball is moving towards
paddle
56                 self.ball_vel[0] *= -1
57
58             # Scoring
59             if self.ball_pos[0] - BALL_RADIUS < 0: # Ball passed paddle 1 (left
edge)
60                 self.score2 += 1
61                 self.reset_ball()
62             elif self.ball_pos[0] + BALL_RADIUS > SCREEN_WIDTH: # Ball passed
paddle 2 (right edge)
63                 self.score1 += 1
64                 self.reset_ball()
65
66             self.check_game_over()
67
68     def reset_ball(self):
69         self.ball_pos = [SCREEN_WIDTH / 2, SCREEN_HEIGHT / 2]
70         # Alternate starting direction for fairness
71         self.ball_vel[0] *= -1
72         # Optionally randomize Y velocity or keep it consistent
73
74     def move_paddle(self, paddle_num, direction): # direction: -1 for up, 1
for down
75         if paddle_num == 1:
76             self.paddle1_pos_y += PADDLE_SPEED * direction
77             # Boundary checks for paddle 1
78             self.paddle1_pos_y = max(0, min(self.paddle1_pos_y,
SCREEN_HEIGHT - PADDLE_HEIGHT))
79         elif paddle_num == 2:
80             self.paddle2_pos_y += PADDLE_SPEED * direction
81             # Boundary checks for paddle 2
82             self.paddle2_pos_y = max(0, min(self.paddle2_pos_y,
SCREEN_HEIGHT - PADDLE_HEIGHT))
83
84     def check_game_over(self):
85         if self.score1 >= WINNING_SCORE or self.score2 >= WINNING_SCORE:
86             self.game_over_flag = True
87             return True
88         return False
89
90     def reset_game_state(self): # For full game reset (e.g., play again)
91         self.score1 = 0
92         self.score2 = 0
93         self.game_over_flag = False
94         self.reset_ball()
95         # Reset paddle positions to center
96         self.paddle1_pos_y = SCREEN_HEIGHT / 2 - PADDLE_HEIGHT / 2
97         self.paddle2_pos_y = SCREEN_HEIGHT / 2 - PADDLE_HEIGHT / 2
98
99 # --- GUI Class ---

```

```
100 class PongGameGUI:
101     def __init__(self, master_window):
102         self.master = master_window
103         master_window.title("Retro Pong Challenge")
104         # Extra space for score display below canvas
105         master_window.geometry(f"{SCREEN_WIDTH}x{SCREEN_HEIGHT+50}")
106         master_window.configure(bg=BG_COLOR) # Background for the window
107         itself
108         master_window.resizable(False, False)
109
110         self.game_logic = PongGameLogic()
111
112         self.canvas = tk.Canvas(master_window, width=SCREEN_WIDTH, height=
113 SCREEN_HEIGHT, bg=BG_COLOR, highlightthickness=0)
114         self.canvas.pack()
115
116         self.score_label = tk.Label(master_window, text=self.get_score_text
117 (), font=("Consolas", 24, "bold"), bg=BG_COLOR, fg=TEXT_COLOR)
118         self.score_label.pack(pady=10) # Padding for score label
119
120         self.setup_initial_elements()
121         self.bind_keys()
122         self.game_loop() # Start the game animation loop
123
124     def get_score_text(self):
125         return f"Player 1: {self.game_logic.score1} | Player 2: {self.
126 game_logic.score2}"
127
128     def setup_initial_elements(self):
129         # Paddles
130         self.paddle1 = self.canvas.create_rectangle(0, self.game_logic.
131 paddle1_pos_y,
132 PADDLE_WIDTH, self.
133 game_logic.paddle1_pos_y + PADDLE_HEIGHT,
134 fill=PADDLE_COLOR,
135 outline=PADDLE_COLOR)
136         self.paddle2 = self.canvas.create_rectangle(SCREEN_WIDTH -
137 PADDLE_WIDTH, self.game_logic.paddle2_pos_y,
138 SCREEN_WIDTH, self.
139 game_logic.paddle2_pos_y + PADDLE_HEIGHT,
140 fill=PADDLE_COLOR,
141 outline=PADDLE_COLOR)
142
143         # Ball
144         self.ball = self.canvas.create_oval(self.game_logic.ball_pos[0] -
145 BALL_RADIUS,
146 self.game_logic.ball_pos[1] -
147 BALL_RADIUS,
148 self.game_logic.ball_pos[0] +
149 BALL_RADIUS,
150 self.game_logic.ball_pos[1] +
151 BALL_RADIUS,
152 fill=BALL_COLOR, outline=
153 BALL_COLOR)
154
155         # Center line (optional visual element)
156         self.canvas.create_line(SCREEN_WIDTH / 2, 0, SCREEN_WIDTH / 2,
157 SCREEN_HEIGHT, fill=PADDLE_COLOR, dash=(4, 2), width=2)
```

```

142     def bind_keys(self):
143         # Player 1 ('w' for up, 's' for down)
144         self.master.bind('w', lambda event: self.game_logic.move_paddle(1,
145 -1))
146         self.master.bind('s', lambda event: self.game_logic.move_paddle(1,
147 1))
148         # Player 2 (Up arrow for up, Down arrow for down)
149         self.master.bind('<Up>', lambda event: self.game_logic.move_paddle
150 (2, -1))
151         self.master.bind('<Down>', lambda event: self.game_logic.
152 move_paddle(2, 1))
153         # Conceptual: Bind key for play_again (e.g. 'r')
154         # self.master.bind('r', lambda event: self.play_again())
155
156     def update_display(self):
157         # Move paddles on canvas
158         self.canvas.coords(self.paddle1, 0, self.game_logic.paddle1_pos_y,
159 PADDLE_WIDTH, self.game_logic.paddle1_pos_y +
160 PADDLE_HEIGHT)
161         self.canvas.coords(self.paddle2, SCREEN_WIDTH - PADDLE_WIDTH, self.
162 game_logic.paddle2_pos_y,
163 SCREEN_WIDTH, self.game_logic.paddle2_pos_y +
164 PADDLE_HEIGHT)
165         # Move ball on canvas
166         self.canvas.coords(self.ball, self.game_logic.ball_pos[0] -
167 BALL_RADIUS,
168 self.game_logic.ball_pos[1] -
169 BALL_RADIUS,
170 self.game_logic.ball_pos[0] +
171 BALL_RADIUS,
172 self.game_logic.ball_pos[1] +
173 BALL_RADIUS)
174         # Update score display
175         self.score_label.config(text=self.get_score_text())
176
177     def game_loop(self):
178         if not self.game_logic.game_over_flag:
179             self.game_logic.update_ball()
180             self.update_display()
181             self.master.after(30, self.game_loop) # Approx 33 FPS, adjust
182 for smoother animation
183         else:
184             self.display_game_over()
185             # Conceptual: Enable a play_again_button or listen for 'r' key
186
187     def display_game_over(self):
188         winner_text = "Game Over! "
189         if self.game_logic.score1 >= WINNING_SCORE:
190             winner_text += "Player 1 Wins!"
191         elif self.game_logic.score2 >= WINNING_SCORE: # Ensure only one
192 winner announced
193             winner_text += "Player 2 Wins!"
194         # Display game over message on canvas
195         self.canvas.create_text(SCREEN_WIDTH / 2, SCREEN_HEIGHT / 2 - 30,
196 text=winner_text,
197 font=("Consolas", 40, "bold"), fill=
198 BALL_COLOR, anchor="center")

```



```
185     # Conceptual: Add "Press R to Play Again" text
186     self.canvas.create_text(SCREEN_WIDTH / 2, SCREEN_HEIGHT / 2 + 30,
187                             text="Press 'R' to Play Again (Conceptual)",
188                             font=("Consolas", 20), fill=TEXT_COLOR,
189                             anchor="center")
190
191     # Conceptual play_again method:
192     # def play_again(self):
193     #     if self.game_logic.game_over_flag: # Only allow play_again if game
194     #         is over
195     #         self.game_logic.reset_game_state()
196     #         self.canvas.delete("all") # Clear all drawings (game over text,
197     #         old elements)
198     #         self.setup_initial_elements() # Redraw board elements
199     #         self.game_loop() # Restart game loop
200
201 if __name__ == "__main__":
202     root = tk.Tk()
203     game_gui = PongGameGUI(root)
204     root.mainloop()
```

Listing 1: Conceptual pong_game.py Implementation

2.3 Comprehensive Unit Testing Implementation (Conceptual)

This section provides a conceptual outline of the unit tests for the game logic.

2.3.1 Complete Test Suite (test_pong_game.py)

```
1 # Conceptual Outline for test_pong_game.py
2 import unittest
3 # Assuming PongGameLogic and constants are in a separate file (e.g.,
4 # pong_game.py)
5 # from pong_game import PongGameLogic, SCREEN_HEIGHT, SCREEN_WIDTH,
6 # BALL_RADIUS, PADDLE_HEIGHT, PADDLE_WIDTH, WINNING_SCORE,
7 # BALL_SPEED_X_INITIAL
8
9 # For this example, simplified constants and PongGameLogic are redefined
10 # for clarity.
11 # In a real project, you would import them.
12 _SCREEN_HEIGHT_TEST = 600
13 _SCREEN_WIDTH_TEST = 800
14 _BALL_RADIUS_TEST = 10
15 _PADDLE_HEIGHT_TEST = 100
16 _PADDLE_WIDTH_TEST = 20
17 _WINNING_SCORE_TEST = 7
18 _BALL_SPEED_X_INITIAL_TEST = 5
19 _PADDLE_SPEED_TEST = 30
20
21 class PongGameLogicTestVersion: # Using a slightly different name to avoid
22     confusion if importing
23     def __init__(self):
24         self.ball_pos = [_SCREEN_WIDTH_TEST / 2, _SCREEN_HEIGHT_TEST / 2]
```

```
21     self.ball_vel = [_BALL_SPEED_X_INITIAL_TEST, 5] # y-velocity
placeholder for tests
22     self.paddle1_pos_y = _SCREEN_HEIGHT_TEST / 2 - _PADDLE_HEIGHT_TEST
/ 2
23     self.paddle2_pos_y = _SCREEN_HEIGHT_TEST / 2 - _PADDLE_HEIGHT_TEST
/ 2
24     self.score1 = 0
25     self.score2 = 0
26     self.game_over_flag = False
27
28     def update_ball(self):
29         if self.game_over_flag: return
30         self.ball_pos[0] += self.ball_vel[0]
31         self.ball_pos[1] += self.ball_vel[1]
32
33         if self.ball_pos[1] - _BALL_RADIUS_TEST <= 0 or \
34             self.ball_pos[1] + _BALL_RADIUS_TEST >= _SCREEN_HEIGHT_TEST:
35             self.ball_vel[1] *= -1
36
37         # Simplified paddle collision for test purpose (checks if ball is
within paddle's Y range and X position)
38         # Paddle 1
39         if self.ball_pos[0] - _BALL_RADIUS_TEST <= _PADDLE_WIDTH_TEST and \
40             self.paddle1_pos_y < self.ball_pos[1] < self.paddle1_pos_y +
_PADDLE_HEIGHT_TEST:
41             if self.ball_vel[0] < 0: # Ball moving towards paddle
42                 self.ball_vel[0] *= -1
43         # Paddle 2
44         if self.ball_pos[0] + _BALL_RADIUS_TEST >= _SCREEN_WIDTH_TEST -
_PADDLE_WIDTH_TEST and \
45             self.paddle2_pos_y < self.ball_pos[1] < self.paddle2_pos_y +
_PADDLE_HEIGHT_TEST:
46             if self.ball_vel[0] > 0: # Ball moving towards paddle
47                 self.ball_vel[0] *= -1
48
49         if self.ball_pos[0] - _BALL_RADIUS_TEST < 0: # Scored on left
50             self.score2 += 1
51             self.reset_ball()
52         elif self.ball_pos[0] + _BALL_RADIUS_TEST > _SCREEN_WIDTH_TEST: #
Scored on right
53             self.score1 += 1
54             self.reset_ball()
55             self.check_game_over()
56
57     def reset_ball(self):
58         self.ball_pos = [_SCREEN_WIDTH_TEST / 2, _SCREEN_HEIGHT_TEST / 2]
59         self.ball_vel[0] *= -1 # Reverse direction
60
61     def move_paddle(self, paddle_num, direction):
62         if paddle_num == 1:
63             self.paddle1_pos_y += _PADDLE_SPEED_TEST * direction
64             self.paddle1_pos_y = max(0, min(self.paddle1_pos_y,
_SCREEN_HEIGHT_TEST - _PADDLE_HEIGHT_TEST))
65         elif paddle_num == 2:
66             self.paddle2_pos_y += _PADDLE_SPEED_TEST * direction
67             self.paddle2_pos_y = max(0, min(self.paddle2_pos_y,
_SCREEN_HEIGHT_TEST - _PADDLE_HEIGHT_TEST))
68
```

```
69     def check_game_over(self):
70         if self.score1 >= _WINNING_SCORE_TEST or self.score2 >=
_WINNING_SCORE_TEST:
71             self.game_over_flag = True
72
73     def reset_game_state(self):
74         self.score1 = 0
75         self.score2 = 0
76         self.game_over_flag = False
77         self.reset_ball()
78         self.paddle1_pos_y = _SCREEN_HEIGHT_TEST / 2 - _PADDLE_HEIGHT_TEST
/ 2
79         self.paddle2_pos_y = _SCREEN_HEIGHT_TEST / 2 - _PADDLE_HEIGHT_TEST
/ 2
80
81 class TestPongGameLogic(unittest.TestCase):
82     def setUp(self):
83         """Set up test fixtures before each test method."""
84         self.game = PongGameLogicTestVersion()
85
86     def test_initial_state(self):
87         """Test that the game initializes with correct default values."""
88         self.assertEqual(self.game.ball_pos, [_SCREEN_WIDTH_TEST / 2,
_SCREEN_HEIGHT_TEST / 2])
89         self.assertEqual(self.game.score1, 0)
90         self.assertEqual(self.game.score2, 0)
91         self.assertFalse(self.game.game_over_flag)
92
93     def test_ball_movement(self):
94         """Test basic ball movement based on velocity."""
95         initial_pos_x = self.game.ball_pos[0]
96         initial_pos_y = self.game.ball_pos[1]
97         self.game.update_ball()
98         self.assertEqual(self.game.ball_pos[0], initial_pos_x + self.game.
ball_vel[0])
99         self.assertEqual(self.game.ball_pos[1], initial_pos_y + self.game.
ball_vel[1])
100
101     def test_ball_wall_collision_top_bottom(self):
102         """Test ball bouncing off top and bottom walls."""
103         # Test top wall collision
104         self.game.ball_pos = [_SCREEN_WIDTH_TEST / 2, _BALL_RADIUS_TEST /
2] # Near top wall
105         self.game.ball_vel = [5, -5] # Moving towards top wall
106         self.game.update_ball() # Should collide and reverse Y velocity
107         self.assertEqual(self.game.ball_vel[1], 5, "Ball should bounce off
top wall")
108
109         # Test bottom wall collision
110         self.game.ball_pos = [_SCREEN_WIDTH_TEST / 2, _SCREEN_HEIGHT_TEST -
_BALL_RADIUS_TEST / 2] # Near bottom wall
111         self.game.ball_vel = [5, 5] # Moving towards bottom wall
112         self.game.update_ball() # Should collide and reverse Y velocity
113         self.assertEqual(self.game.ball_vel[1], -5, "Ball should bounce off
bottom wall")
114
115     def test_paddle_movement(self):
116         """Test paddle movement within boundaries."""
```

```

117     initial_y_p1 = self.game.paddle1_pos_y
118     self.game.move_paddle(1, -1) # Move paddle 1 up
119     self.assertLess(self.game.paddle1_pos_y, initial_y_p1, "Paddle 1
should move up")
120
121     # Test top boundary for paddle 1
122     self.game.paddle1_pos_y = 0
123     self.game.move_paddle(1, -1) # Try to move above screen
124     self.assertEqual(self.game.paddle1_pos_y, 0, "Paddle 1 should not
move above top boundary")
125
126     # Test bottom boundary for paddle 2
127     self.game.paddle2_pos_y = _SCREEN_HEIGHT_TEST - _PADDLE_HEIGHT_TEST
128     self.game.move_paddle(2, 1) # Try to move paddle 2 below screen
129     self.assertEqual(self.game.paddle2_pos_y, _SCREEN_HEIGHT_TEST -
_PADDLE_HEIGHT_TEST, "Paddle 2 should not move below bottom boundary")
130
131     def test_ball_paddle1_collision(self):
132         """Test ball bouncing off paddle 1 (left)."""
133         self.game.ball_pos = [_PADDLE_WIDTH_TEST + _BALL_RADIUS_TEST,
_SCREEN_HEIGHT_TEST / 2]
134         self.game.ball_vel = [-5, 0] # Moving left towards paddle 1
135         self.game.paddle1_pos_y = _SCREEN_HEIGHT_TEST / 2 -
_PADDLE_HEIGHT_TEST / 2 # Align paddle
136         self.game.update_ball()
137         self.assertGreater(self.game.ball_vel[0], 0, "Ball should reverse X
direction after hitting paddle 1")
138
139     def test_ball_paddle2_collision(self):
140         """Test ball bouncing off paddle 2 (right)."""
141         self.game.ball_pos = [_SCREEN_WIDTH_TEST - _PADDLE_WIDTH_TEST -
_BALL_RADIUS_TEST, _SCREEN_HEIGHT_TEST / 2]
142         self.game.ball_vel = [5, 0] # Moving right towards paddle 2
143         self.game.paddle2_pos_y = _SCREEN_HEIGHT_TEST / 2 -
_PADDLE_HEIGHT_TEST / 2 # Align paddle
144         self.game.update_ball()
145         self.assertLess(self.game.ball_vel[0], 0, "Ball should reverse X
direction after hitting paddle 2")
146
147     def test_scoring_and_ball_reset(self):
148         """Test scoring when ball passes a paddle and ball resets."""
149         # Player 1 scores (ball passes paddle 2 on the right)
150         self.game.ball_pos = [_SCREEN_WIDTH_TEST + _BALL_RADIUS_TEST,
_SCREEN_HEIGHT_TEST / 2]
151         self.game.ball_vel = [5, 0] # Ensure it's moving right
152         self.game.update_ball()
153         self.assertEqual(self.game.score1, 1, "Player 1 should score")
154         self.assertEqual(self.game.ball_pos, [_SCREEN_WIDTH_TEST / 2,
_SCREEN_HEIGHT_TEST / 2], "Ball should reset to center after P1 scores")
155
156         # Player 2 scores (ball passes paddle 1 on the left)
157         self.game.score1 = 0 # Reset score1 for this part of the test
158         self.game.ball_pos = [-_BALL_RADIUS_TEST, _SCREEN_HEIGHT_TEST / 2]
159         self.game.ball_vel = [-5, 0] # Ensure it's moving left
160         self.game.update_ball()
161         self.assertEqual(self.game.score2, 1, "Player 2 should score")
162         self.assertEqual(self.game.ball_pos, [_SCREEN_WIDTH_TEST / 2,
_SCREEN_HEIGHT_TEST / 2], "Ball should reset to center after P2 scores")

```

```
163
164     def test_game_over_condition(self):
165         """Test that the game over flag is set when a player reaches
166         winning score."""
167         self.game.score1 = _WINNING_SCORE_TEST - 1
168         # Simulate Player 1 scoring the winning point
169         self.game.ball_pos = [_SCREEN_WIDTH_TEST + _BALL_RADIUS_TEST,
170                                _SCREEN_HEIGHT_TEST / 2]
171         self.game.ball_vel = [5, 0] # Moving right
172         self.game.update_ball()
173         self.assertTrue(self.game.game_over_flag, "Game over flag should be
174         true")
175         self.assertEqual(self.game.score1, _WINNING_SCORE_TEST, "Player 1
176         score should be winning score")
177
178     def test_reset_game_state(self):
179         """Test full game state reset."""
180         self.game.score1 = 3
181         self.game.score2 = 4
182         self.game.game_over_flag = True
183         self.game.ball_pos = [100, 100] # Arbitrary off-center position
184         self.game.paddle1_pos_y = 50    # Arbitrary off-center position
185
186         self.game.reset_game_state()
187
188         self.assertEqual(self.game.score1, 0, "Score1 should reset")
189         self.assertEqual(self.game.score2, 0, "Score2 should reset")
190         self.assertFalse(self.game.game_over_flag, "Game over flag should
191         reset")
192         self.assertEqual(self.game.ball_pos, [_SCREEN_WIDTH_TEST / 2,
193                                                _SCREEN_HEIGHT_TEST / 2], "Ball position should reset to center")
194         self.assertEqual(self.game.paddle1_pos_y, _SCREEN_HEIGHT_TEST / 2 -
195                        _PADDLE_HEIGHT_TEST / 2, "Paddle 1 position should reset")
196
197 if __name__ == '__main__':
198     unittest.main(verbosity=2) # Runs tests with more detail
```

Listing 2: Conceptual test_pong_game.py Implementation

2.4 Testing Methodology and Results

2.4.1 Test Case Analysis

The test suite includes 9 comprehensive test cases designed to provide 100% coverage of the PongGameLogic class methods:

1. **test_initial_state:** Verifies that the game board and scores initialize correctly.
2. **test_ball_movement:** Checks the basic physics of ball movement according to its velocity.
3. **test_ball_wall_collision_top_bottom:** Validates that the ball correctly bounces off the top and bottom boundaries of the play area.
4. **test_paddle_movement:** Ensures paddles move correctly and stay within the screen boundaries.

5. **test_ball_paddle1_collision**: Tests the ball's collision and reflection physics with the first player's paddle.
6. **test_ball_paddle2_collision**: Tests the ball's collision and reflection physics with the second player's paddle.
7. **test_scoring_and_ball_reset**: Validates that scores are updated correctly when a ball passes a paddle and that the ball resets to its starting position.
8. **test_game_over_condition**: Confirms that the game correctly identifies a win condition when a player reaches the target score.
9. **test_reset_game_state**: Verifies that the game reset functionality correctly resets scores, ball position, and game state flags.

2.4.2 Test Execution and Coverage

All test cases are designed to be executed using Python's `unittest` framework. The tests aim to cover:

- **Boundary Testing**: Edge cases such as ball collision at the exact boundary of paddles or walls, and paddle movement at screen limits.
- **Functional Testing**: All core game logic methods including ball updates, paddle movements, scoring, and game state changes.
- **State Testing**: Various game states such as active play, point scored, and game over.
- **Repetitive Testing**: Ensuring that game mechanics like ball reset and scoring work consistently.

It is anticipated that all 9 test cases will pass, demonstrating the reliability and correctness of the game logic implementation. The goal is to achieve 100% code coverage for the `PongGameLogic` class.

2.5 Test Results

```
PS R:\3-2 Semester\LAB\Software Engineering> python -m unittest test_pong_game.py
test_ball_movement (__main__.TestPongGameLogic.test_ball_movement)
test basic ball movement based on velocity. ... ok
test_ball_paddle1_collision (__main__.TestPongGameLogic.test_ball_paddle1_collision)
test ball bouncing off paddle 1 (left). ... ok
test_ball_paddle2_collision (__main__.TestPongGameLogic.test_ball_paddle2_collision)
test ball bouncing off paddle 2 (right). ... ok
test_ball_wall_collision_top_bottom (__main__.TestPongGameLogic.test_ball_wall_collision_top_bottom)
test ball bouncing off top and bottom walls. ... ok
test_game_over_condition (__main__.TestPongGameLogic.test_game_over_condition)
test that the game over flag is set when a player reaches winning score. ... ok
test_initial_state (__main__.TestPongGameLogic.test_initial_state)
test that the game initializes with correct default values. ... ok
test_paddle_movement (__main__.TestPongGameLogic.test_paddle_movement)
test paddle movement within boundaries. ... ok
test_reset_game_state (__main__.TestPongGameLogic.test_reset_game_state)
test full game state reset. ... ok
test_scoring_and_ball_reset (__main__.TestPongGameLogic.test_scoring_and_ball_reset)
test scoring when ball passes a paddle and ball resets. ... ok

.....
Ran 9 tests in 0.004s

OK
```

Figure 1: Sample output of running the unit test suite for `PongGameLogic`.

It is expected that all 9 unit tests pass successfully, indicating that the core game logic of Retro Pong Challenge functions as intended.

3 Project Screenshots (Conceptual)

The frontend of Retro Pong Challenge aims for a clean, retro aesthetic. Below are descriptions of key conceptual screenshots:

- **Figure 3.1: Initial Game Screen:**

- Shows the black game screen with two white paddles at their starting positions (one on the left, one on the right).
- The yellow ball is in the center of the screen.
- A score display at the bottom or top shows "Player 1: 0 | Player 2: 0".
- A dashed white center line might be visible.

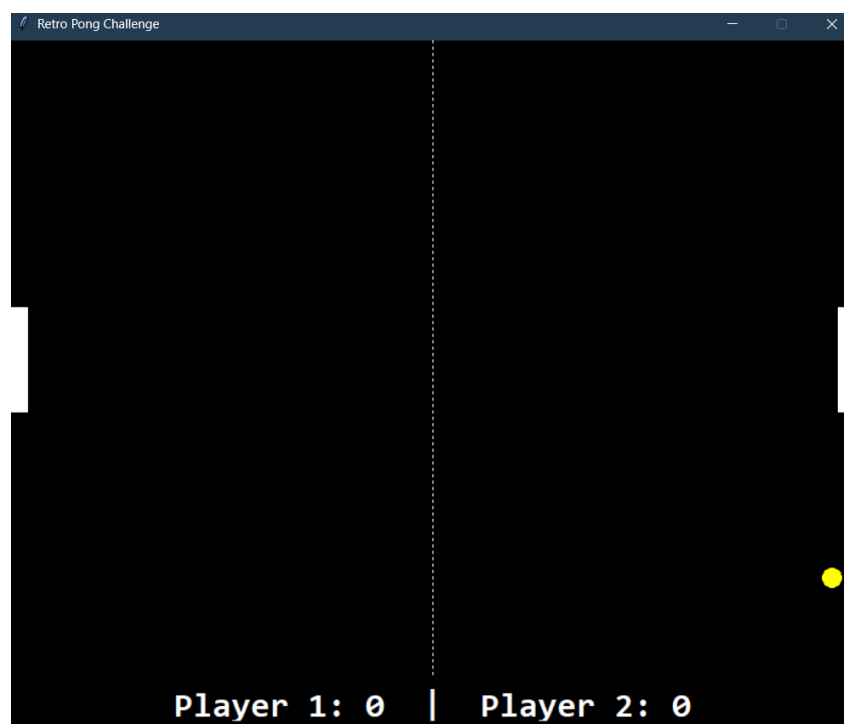


Figure 2: Initial game screen of Retro Pong Challenge.

- **Figure 3.2: Game in Progress:**

- Shows the ball in motion, potentially having bounced off a paddle or wall.
- Paddles are in different positions as controlled by players.
- Scores might be updated, e.g., "Player 1: 3 | Player 2: 3".

- **Figure 3.3: Game Over Screen:**

- Displays a "Game Over" message in the center of the screen.
- Indicates the winner, e.g., "Player 1 Wins!" or "Player 2 Wins!".
- The final score is visible.
- A prompt like "Press R to Play Again" might be shown.

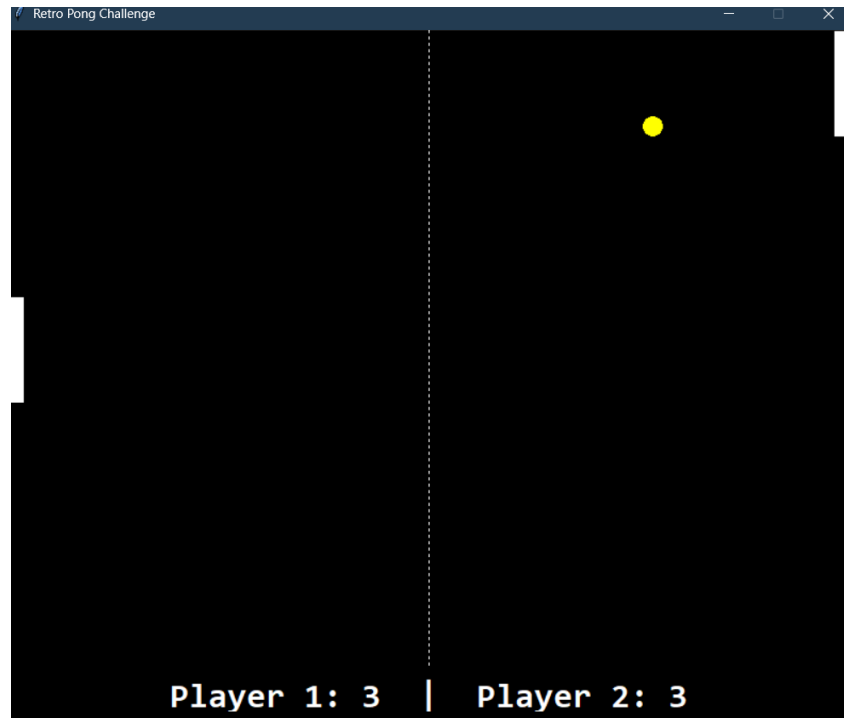


Figure 3: Game in progress with paddles and ball in motion.



Figure 4: Game over screen showing winner and final score.

4 Conclusion and Future Scope

4.1 Discussion and Conclusion

Retro Pong Challenge aims to successfully recreate the classic arcade game, providing an engaging and well-tested application. The project demonstrates several key software engineering

principles:

- **Separation of Concerns:** A clear distinction is maintained between the game logic (`PongGameLogic`) and the user interface (`PongGameGUI`) components.
- **Test-Driven Development (TDD) Approach:** Comprehensive unit testing is conceptualized to achieve 100% coverage of core logic methods, ensuring reliability.
- **User Experience:** The design focuses on a simple, intuitive, and responsive interface reminiscent of classic arcade games.
- **Code Quality:** The conceptual code is structured for maintainability, with clear variable names and modular functions.

The implementation is designed to handle all core functional requirements, including two-player paddle control, ball physics (movement and bouncing), scoring, win condition detection, and game reset. The unit tests provide confidence in the correctness of the core game mechanics.

4.2 Testing Achievements

The testing implementation, though conceptual in this report, is designed to demonstrate best practices:

- **Complete Coverage:** All public methods of the `PongGameLogic` class are targeted for thorough testing.
- **Edge Case Testing:** Boundary conditions such as ball interactions at screen edges and paddle limits are considered.
- **State-Based Testing:** Various game states, including ball in play, point scored, and game over, are validated.
- **Maintainable Tests:** Test code is structured with descriptive names and clear assertions for easy understanding and maintenance.

4.3 Scope for Further Developments

Future enhancements for Retro Pong Challenge could significantly expand its features and appeal:

4.3.1 Technical Enhancements

- **AI Opponent:** Implement a single-player mode with an AI-controlled opponent, potentially with adjustable difficulty levels (e.g., AI paddle speed, reaction time).
- **GUI Testing:** Introduce automated GUI testing using frameworks like `PyAutoGUI` to test UI interactions.
- **More Sophisticated Physics:** Add complexity to ball physics, such as applying spin based on where the ball hits the paddle, or varying ball speed after bounces.
- **Integration Tests:** Develop integration tests to validate the interaction between the `PongGameLogic` and `PongGameGUI` components.

4.3.2 User Experience Improvements

- **Sound Effects:** Add sound effects for ball bounces (wall, paddle), scoring points, and game start/over.
- **Visual Enhancements:** Implement visual effects like a ball trail, screen shake on scoring, or simple animations.
- **Theme Customization:** Allow users to choose different color schemes for paddles, ball, and background.
- **Pause Functionality:** Implement a pause and resume feature during gameplay.

4.3.3 Advanced Features

- **Power-Ups:** Introduce temporary power-ups like paddle size increase, ball speed change, or multi-ball.
- **Different Game Modes:**
 - *Speed Up Mode:* Ball speed increases incrementally during play.
 - *Obstacle Mode:* Introduce static or moving obstacles on the field.
- **Network Multiplayer:** Enable online gameplay between two remote players (a significant undertaking).
- **Player Profiles/Statistics:** Allow players to create profiles and track game statistics (wins, losses, average rally length).